Bonus — Streamlit: Turning This Course's Static Figures Into Interactive Demos

Goal today: every result so far has been a static matplotlib figure baked into a PDF. That's the right format for *reporting* a result, but the wrong format for *exploring* one — "what if the learning rate were higher?" shouldn't require editing a script and re-running it. Streamlit closes that gap: plain Python functions become a live, browser-based app with almost no new syntax to learn.

Code: `app.py`. **Run it** (after the repo-root `Setup` in the main `README.md`):

```
cd 04b_streamlit_demo
streamlit run app.py
```

Opens at `http://localhost:8501` by default.

Where this sits in the course: a bonus lesson between Days 4 and 5, not one of the 15 numbered days — it doesn't introduce new deep learning theory, only a new way to interact with models you've already built. No later day depends on it.

---

1. The core mental model: a script that reruns top to bottom

A Streamlit app is an ordinary Python script. The entire trick is one fact: **every time the user interacts with a widget (moves a slider, clicks a button), Streamlit reruns the *entire script* from top to bottom**, not just a callback function. `st.slider(...)` returns the widget's *current* value directly, inline, as if it were a plain variable — there's no separate event-handler wiring to write.

This has one immediate, important consequence: **anything expensive (training a model, loading a large file) needs explicit caching**, or it re-runs on every single widget interaction, including ones unrelated to it. This app uses two caching decorators for two different needs:

- **`@st.cache_data`** (Tab 2, `run_optimizer_paths`) — caches a function's *return value*, keyed on its arguments. Move a learning-rate slider back to a value you've already tried, and Streamlit returns the cached result instantly instead of recomputing it. Use this for functions that return plain data (arrays, dicts).
- **`@st.cache_resource`** (Tab 3, `train_demo_model`) — caches a *live object* (here, a trained `nn.Module`) once, shared across the whole app session, regardless of what any widget does afterward. Use this for things you want built exactly once (a model, a database connection) — `cache_data` would try to pickle/copy the model on every cache hit, which is both wasteful and, for some objects, not even possible.

2. What each tab reuses, and what's new

Tab 1 (Perceptron vs MLP, Days 3-4): identical training loop to `03_perceptron_gradient_descent/code/train_perceptron.py` and `04_mlp_backpropagation/code/activations_and_vanishing_gradients.py`, now driven by `st.selectbox` / `st.radio` / `st.slider` instead of hardcoded arguments, wrapped in `st.session_state` so the last trained result survives across reruns triggered by *other* widgets (moving a slider elsewhere doesn't blank the plot until you explicitly click Train again).

Tab 2 (Optimizer race, Day 5): identical loss surface and optimizer setup to `05_losses_and_optimizers/code/optimizer_race.py`. The new idea here is a **scrubber**: instead of always plotting the full 60-step path, an `st.slider` controls how many steps are drawn, so you can watch SGD's oscillation build up step by step rather than only seeing the final tangled line.

Tab 3 (Shape classifier + Grad-CAM, Day 15): a deliberately *lighter* version of the capstone's residual CNN (smaller, fewer epochs) — trading some accuracy for a training time short enough to run once, live, the first time the app loads, via `@st.cache_resource`, rather than requiring a pre-saved checkpoint file. Same Grad-CAM mechanism as `15_capstone_project/code/capstone_pipeline.py`'s `grad_cam()`, applied to a freshly-generated random shape every time you click the button.

3. A real bug this caught — worth knowing as a general lesson

The first version of `app.py` used `Path(__file__).parents[1]` to locate the course root for `sys.path` (the same pattern every other day's `code/` script uses). Testing it with Streamlit's `AppTest` API (§4) failed immediately:

```
IndexError: 1
  ... Path(__file__).parents[1]
```

Why: `Path(__file__).parents[1]` only works reliably when `__file__` is already an *absolute* path — `parents[0]` is the immediate parent, `parents[1]` the one above it. If `__file__` is relative (as it was in the `AppTest` harness's execution context — and, in principle, could be under other launch methods too, depending on how the script is invoked relative to the current working directory), `.parents` may only have as many entries as the relative path has components, and indexing past that raises exactly this error. The fix is one call: **`Path(__file__).resolve().parents[1]`** — `.resolve()` converts to an absolute path first, making the parent chain always long enough regardless of how the script was launched. This is a good general habit for exactly this `sys.path.insert` pattern anywhere it's used, not just here — it's the more robust form for very little extra cost.

4. Testing a Streamlit app without a browser

You don't need to click through an app by hand to verify it works. `streamlit.testing.v1.AppTest` runs the script headlessly and gives you programmatic access to every widget and any exception raised:

```
from streamlit.testing.v1 import AppTest
at = AppTest.from_file("app.py", default_timeout=90)
at.run()
assert not at.exception          # catches the bug from §3 directly

train_button = [b for b in at.button if b.label == "Train"][0]
train_button.click().run()       # simulates the click, reruns the script
assert not at.exception
```

This is what actually caught the `§3` bug — a plain `curl` request to the running server returned a healthy `HTTP 200` even *with* the bug present in the un-hit code path, because Streamlit's initial page load doesn't necessarily execute a widget-driven branch the same way a full script rerun after interaction does. `AppTest` executes the real script logic and surfaces the real exception, which is a meaningfully stronger check than "the server responds."

Library notes: Streamlit

- **Layout**: `st.tabs([...])` for the top-level tab structure used here; `st.columns([1, 2])` (used inside each tab) splits horizontal space by the given ratio — controls on the narrow left column, plots on the wider right, a common dashboard layout.
- **`st.pyplot(fig)`** renders a matplotlib figure directly — every plot in this app reuses the exact same `matplotlib` code style as the rest of the course (`fig, ax = plt.subplots(...)`), just displayed inline in the browser instead of saved to a PNG file.
- **`st.session_state`** is a dict-like object that *persists across reruns* within one browser session — the only place to store state that should survive a rerun triggered by an unrelated widget (Tab 1's trained-model result, Tab 3's shape-generation counter). Reading an uninitialized key raises `KeyError`, so this app always checks `if "key" not in st.session_state` before first use.
- **`st.metric(label, value, delta=...)`** — a small stat display with optional colored delta indicator; `delta_color="inverse"` (used in Tab 3) flips red/green so a "WRONG" delta shows as an alert color instead of Streamlit's default (which would otherwise color any non-empty delta green).
- **Deployment note, not covered hands-on here**: `streamlit run app.py` is a local dev server. Sharing an app with others normally means either Streamlit Community Cloud (free, connects to a GitHub repo) or containerizing it behind a reverse proxy — out of scope for this bonus lesson, but worth knowing the path exists once a demo is worth sharing beyond your own machine.

Exercises

1. Add a fourth tab reusing Day 9's sine-wave RNN prediction (`make_sine_sequence`) — a slider for how many timesteps to predict ahead, plotting predicted vs. true continuation.
2. In Tab 1, add a checkbox "show gradient norm" that runs the training loop once more with `retain_grad()` (Day 4 pattern) and plots per-epoch gradient norm alongside the loss curve.
3. Deliberately reintroduce the `parents[1]` bug from §3, then write your own `AppTest`-based check that would have caught it — this is the fastest way to internalize why `.resolve()` matters here.

Back to the main sequence: Day 5 (`../05_losses_and_optimizers/`) — loss functions and optimizers, covered in the notes this bonus lesson's Tab 2 made interactive.